

Crash Course in Supercomputing



Source:

<https://www.nersc.gov/news-and-events/calendar-of-events/hpc-crash-course-jun2025>

<https://www.cac.cornell.edu/education/training/StampedeJan2017/IntroOpenMPandMPIwithKNL.pdf>

Topics

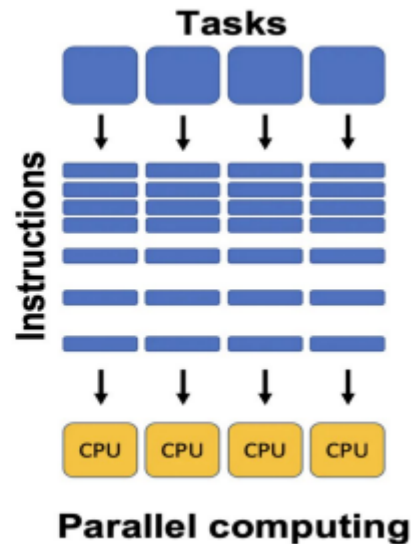
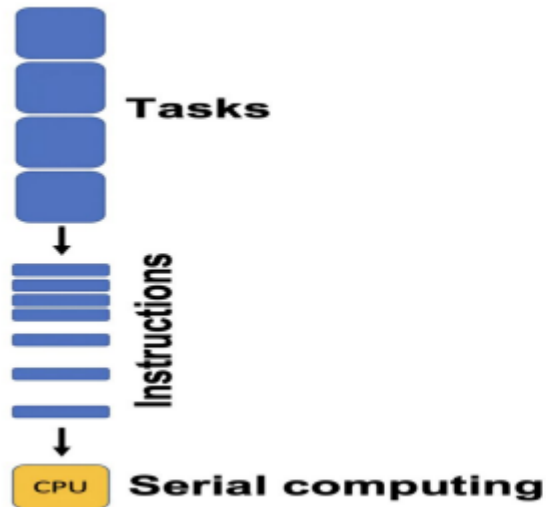
- Introduction to Parallel Programming Concepts
- Understanding Supercomputer Architecture
- Basic Parallelism & MPI
 - MPI Collectives
- Introduction to OpenMP

What is Parallelism?

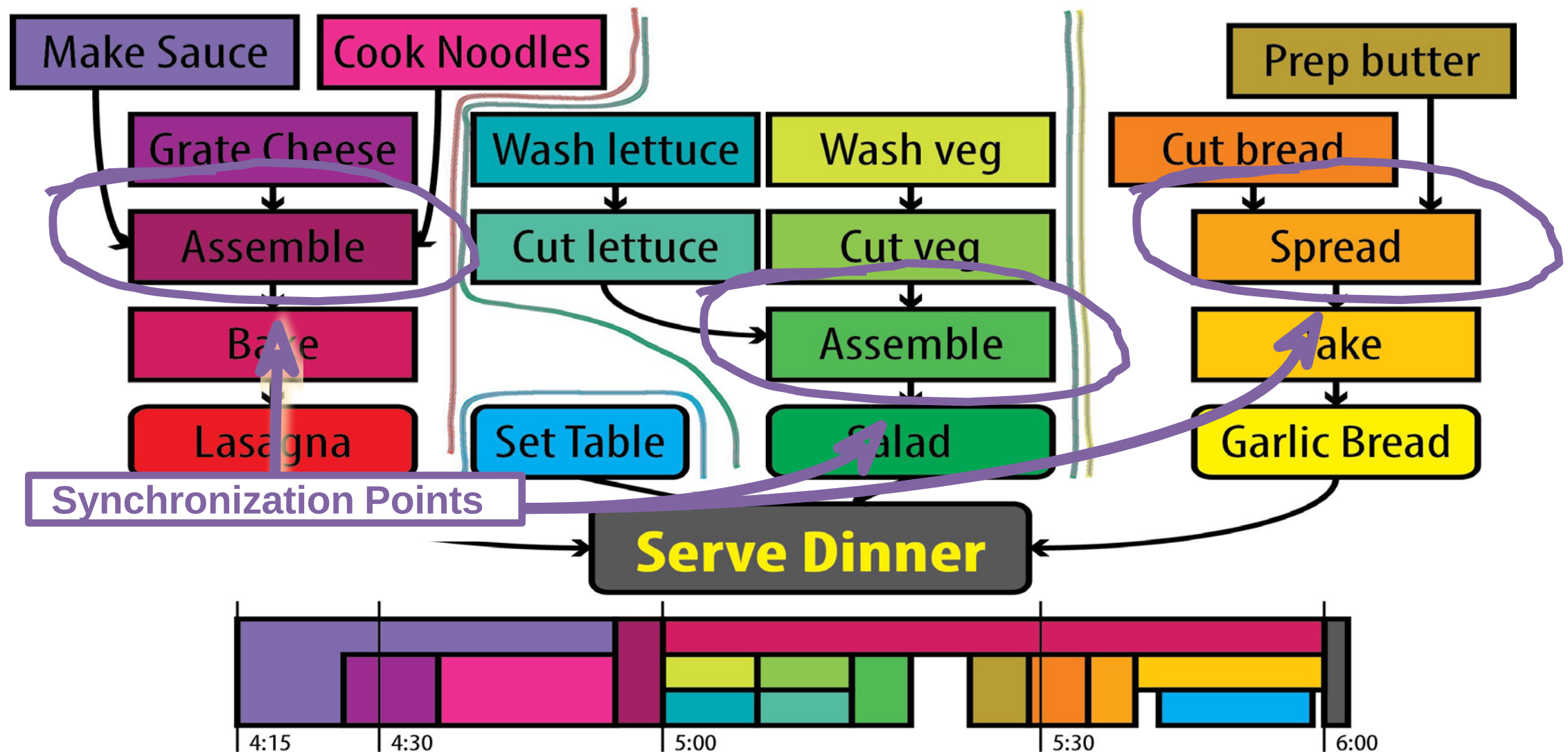
- How?
 - the concept of dividing a task or problem into smaller subtasks that can be **executed simultaneously**.
- Benefit?
 - Work can get done more efficiently, thus quicker

Serial vs. Parallel

- Serial: tasks must be performed in sequence
- Parallel: tasks can be performed independently in any order



Serial vs. Parallel: Graph



Could have several chefs, each performing one parallel task. This is concept behind parallel computing.

Discussion: Jigsaw Puzzle



- Suppose we want to do a large, N -piece jigsaw puzzle (e.g., $N = 10,000$ pieces)
 - Time for one person to complete puzzle: T hours
 - How can we decrease walltime to completion?
- Impact of having **multiple people at the table**
 - Walltime to completion
 - Communication
 - Resource contention
- Let number of people = p
 - Think about what happens when $p = 1, 2, 4, \dots 5000$

Discussion: Jigsaw Puzzle

- Alternate setup: p people, each at separate table with N/p pieces each
- What is the impact on
 - Waltime to completion
 - Communication
 - Resource contention?
- Alternate setup: divide puzzle by features, each person works on one feature, e.g., mountain, sky, stream, tree, meadow, etc.
- What is the impact on
 - Waltime to completion
 - Communication
 - Resource contention?

Parallel Algorithm Design: PCAM

- Partition
 - Decompose problem into fine-grained tasks to maximize potential parallelism
- Communication
 - Determine communication pattern among tasks
- Agglomeration
 - Combine into coarser-grained tasks, if necessary, to reduce communication requirements or other costs
- Mapping
 - Assign tasks to processors, subject to tradeoff between communication cost and concurrency

What Is a Supercomputer?

- “The biggest, fastest computer right this minute.” – Henry Neeman
[Tips on Identifying a Supercomputer](#)
- Generally, at least **100 times** more powerful than PC
- This field of study known as supercomputing, high-performance computing (HPC), or scientific computing
- Scientists utilize supercomputers to solve complex problems.
- Really hard problems need really LARGE (super)computers

Supercomputing Architectures

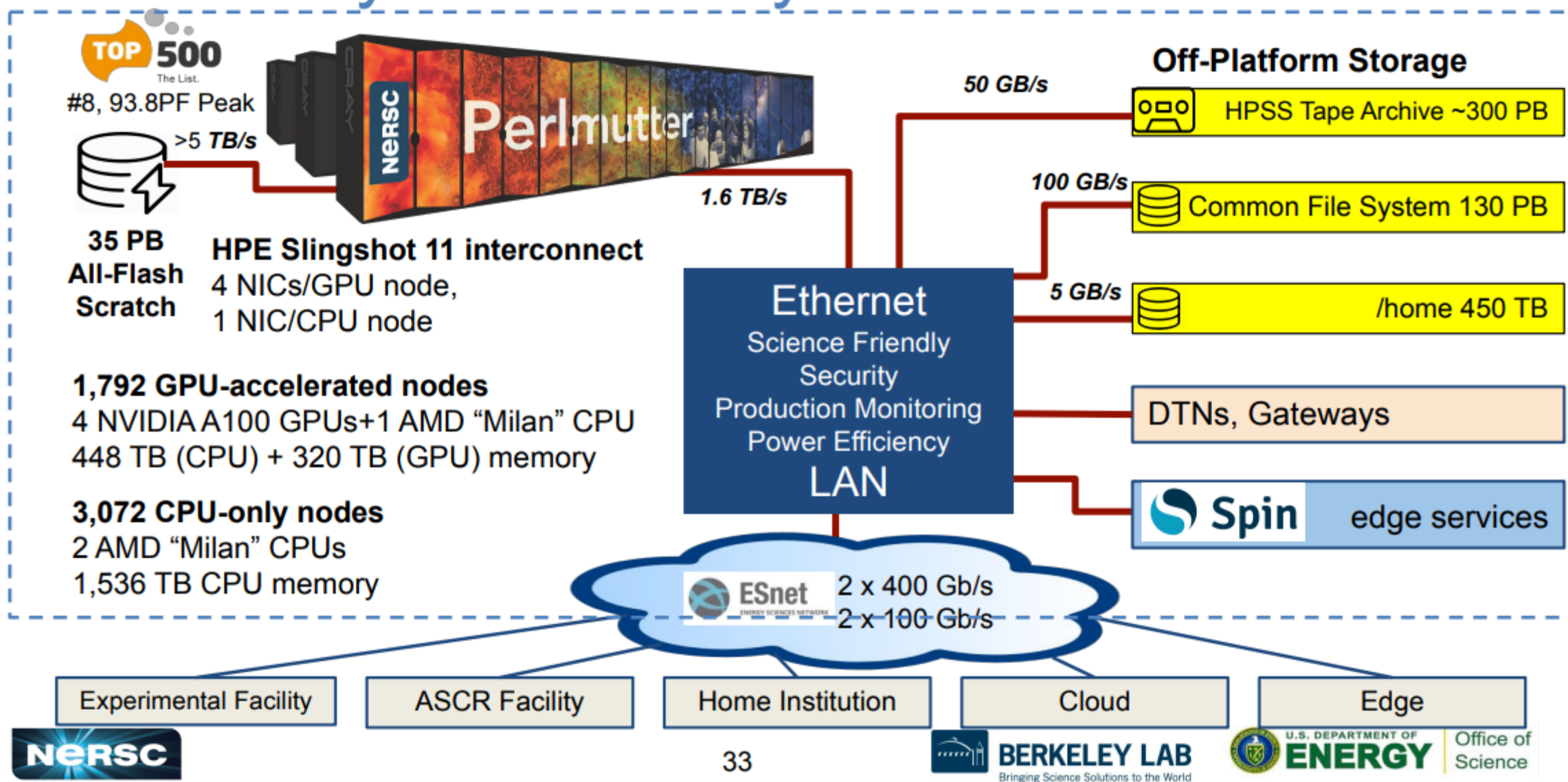
- Cluster Architecture
 - Connects multiple standalone computers to work together as a single system. Provides a cost-effective solution for scalable computing power.
- Symmetric Multiprocessing (SMP)
 - Involves multiple processors sharing a single memory space. Suitable for tasks requiring frequent communication between processors.
- Massively Parallel Processing (MPP)
 - Consists of many processors with their own memory. Effective for tasks that can be divided into independent subtasks.

State-of-the-Art Architectures

Today: hybrid architectures very common

- Multiple {32, 64, 68, 128, 256}-core nodes, connected to other nodes by (slow) interconnect
- Cores in node share memory (like small SMP machines)
- Machine appears to follow cluster architecture (with multi-core nodes rather than single processors)
- To take advantage of all parallelism, use MPI (cluster) and OpenMP (SMP) hybrid programming

NERSC Systems Ecosystem



Introduction to Message Passing Interface (MPI)

- Industry standard for parallel programming for **distributed memory systems**
 - Each process has its own address space and no way to get at another's
 - MPI-1 was released in 1994
- MPI implemented by many vendors
 - OpenMPI (open source)
- MPI function library is used in writing C, C++, or Fortran programs in HPC

The Six Necessary MPI Commands

1. `int MPI_Init(int *argc, char **argv)`
2. `int MPI_Finalize(void)`
3. `int MPI_Comm_size(MPI_Comm comm, int *size)`
4. `int MPI_Comm_rank(MPI_Comm comm, int *rank)`
5. `int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)`
6. `int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)`

Initiation and Termination

- **MPI_Init(int *argc, char **argv)** initiates MPI
 - Place in body of code **after variable declarations and before any MPI commands**
 - Initializes the MPI execution environment.
 - Must be called before any other MPI function.
- **MPI_Finalize(void)** shuts down MPI
 - Place near end of code, after last MPI command
 - Terminates the MPI execution environment.
 - No MPI function can be called after this except MPI_Init and MPI_Finalize.

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>
```

```
int main(int argc, char **argv)
{
```

```
    char message[20];
    int i, rank, size, tag = 99;
    MPI_Status status;
```

```
    MPI_Init(&argc, &argv)
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    if (rank == 0)
```

```
    {
```

```
        strcpy(message, "Hello, world");
```

```
        for (i = 1; i < size; i++)
```

```
            MPI_Send(message, 13, MPI_CHAR, i, tag, MPI_COMM_WORLD);
```

```
    }
```

```
    else
```

```
        MPI_Recv(message, 20, MPI_CHAR, 0, tag, MPI_COMM_WORLD, &status);
```

```
    printf("Message from process %d : %s\n", rank, message);
```

```
    MPI_Finalize();
```

```
}
```

Initialize MPI environment

Nearly all MPI functions must be called after MPI_Init and before MPI_Finalize.

Some implementations may also use MPI_Init to make "main" arguments argc and argv available to all tasks.

Close MPI environment

Environmental Inquiry

- **MPI_Comm_size(MPI_Comm comm, int *size)**
 - Determines the size of the group associated with a communicator
 - Allows flexibility in number of processes used in program
- **MPI_Comm_rank(MPI_Comm comm, int *rank)**
 - Find out identifier of current process
 - Determines the rank of the calling process in the communicator.
 - $0 \leq \text{rank} \leq \text{size}-1$

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>
```

```
int main(int argc, char **argv)
```

```
{
```

```
    char message[20];
```

```
    int i, rank, size, tag = 99;
```

```
    MPI_Status status;
```

```
    MPI_Init(&argc, &argv)
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    if (rank == 0)
```

```
    {
```

```
        strcpy(message, "Hello, world");
```

```
        for (i = 1; i < size; i++)
```

```
            MPI_Send(message, 13, MPI_
```

```
    }
```

```
    else
```

```
        MPI_Recv(message, 20, MPI_CH
```

```
        printf( "Message from process %d :
```

```
        MPI_Finalize();
```

```
    }
```

Returns number of processes

Input: communicator name.

Output: communicator size.

MPI_COMM_WORLD is the default communicator.

Returns the rank, or process number

Input: communicator name.

Output: rank of this process within that communicator.

MPI_COMM_WORLD is the default communicator.

```
dong@pluto2:~/public_html/cs471/lecture/hpc$ mpicc test-openmpi.c -o test_openmpi
dong@pluto2:~/public_html/cs471/lecture/hpc$ mpirun -np 8 test-openmpi
Hello from rank 7 of 8
Hello from rank 2 of 8
Hello from rank 0 of 8
Hello from rank 1 of 8
Hello from rank 4 of 8
Hello from rank 5 of 8
Hello from rank 6 of 8
Hello from rank 3 of 8
dong@pluto2:~/public_html/cs471/lecture/hpc$ vi test-openmpi.c
dong@pluto2:~/public_html/cs471/lecture/hpc$ cat test-openmpi.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("Hello from rank %d of %d\n", rank, size);
    MPI_Finalize();
    return 0;
}dong@pluto2:~/public_html/cs471/lecture/hpc$ |
```

Message Passing: Send and Receive

- **MPI_Send(const void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)**
 - Performs a send from this MPI process to another.
 - Send message of length **count** items and datatype **datatype** contained in **buf** with tag **tag** to process number **dest** in communicator **comm**.
 - E.g. MPI_Send(&x, 1, MPI_DOUBLE, manager, me, MPI_COMM_WORLD)
- **MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)**
 - Performs a blocking receive of data from another process.
 - Receive message of length **count** items and datatype **datatype** with tag **tag** in buffer **buf** from process number **source** in communicator **comm**, and record status **status**
 - E.g. MPI_Recv(&x, 1, MPI_DOUBLE, source, source, MPI_COMM_WORLD, &status)

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>

int main(int argc, char **argv)
{
    char message[20];
    int i, rank, size, tag = 99;
    MPI_Status status;

    MPI_Init(&argc, &argv)
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0)
    {
        strcpy(message, "Hello, world!");
        for (i = 1; i < size; i++)
            MPI_Send(message, 13, MPI_CHAR, i, tag, MPI_COMM_WORLD);
    }
    else
        MPI_Recv(message, 20, MPI_CHAR, 0, tag, MPI_COMM_WORLD,
        &status);

    printf("Message from process %d\n", rank);

    MPI_Finalize();
}
```

Send a message

Blocking send of data in the buffer.

Receive a message

Blocking receive of data into the buffer.

Messages Three Parameters Describe the Data

`MPI_Send( message, 13, MPI_CHAR, i, tag, MPI_COMM_WORLD );`

`MPI_Recv( message, 20, MPI_CHAR, 0, tag, MPI_COMM_WORLD, &status);`

Type of data, should be same
for send and receive
MPI_Datatype type

Number of elements (items, not bytes)
Recv number should be greater than or
equal to amount sent
int count

Address where the data start
void data*

Messages Three Parameters Specify Routing

```
MPI_Send( message, 13, MPI_CHAR, i, tag, MPI_COMM_WORLD );
```

```
MPI_Recv( message, 20, MPI_CHAR, 0, tag, MPI_COMM_WORLD, &status);
```

Identify process you're communicating with by rank number
int dest/src

Arbitrary tag number, must match up (receiver can specify MPI_ANY_TAG to indicate that any tag is acceptable)
int tag

Communicator specified for send and receive must match, no wildcards
MPI_Comm comm

Returns information on received message
MPI_Status status*

```
dong@pluto2:~/public_html/cs471/lecture/hpc$ mpicc test-openmpi-1.c -o test-openmpi-1
dong@pluto2:~/public_html/cs471/lecture/hpc$ mpirun -np 4 test-openmpi-1
Message from process 0 : Hello, world!
Message from process 2 : Hello, world!
Message from process 3 : Hello, world!
Message from process 1 : Hello, world!
dong@pluto2:~/public_html/cs471/lecture/hpc$ cat test-openmpi-1.c
#include <stdio.h>
#include <string.h>
#include <mpi.h>

int main(int argc, char** argv)
{
    char message[20];
    int i, rank, size, tag = 99;

    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == 0)
    {
        strcpy(message, "Hello, world!");
        for (i = 1; i < size; i++)
            MPI_Send(message, 13, MPI_CHAR, i, tag, MPI_COMM_WORLD);
    }else{
        MPI_Recv(message, 20, MPI_CHAR, 0, tag, MPI_COMM_WORLD, &status);
    }

    printf( "Message from process %d : %.13s\n", rank, message);

    MPI_Finalize();

    return 0;
}
```



```
dong@pluto2:~/public_html/cs471/lecture/hpc$ mpirun -np 4 test-openmpi-2
```

```
Sent 0 to proc 1, received 1 from proc 1
```

```
Sent 1 to proc 0, received 0 from proc 0
```

```
Sent 2 to proc 3, received 3 from proc 3
```

```
Sent 3 to proc 2, received 2 from proc 2
```

```
dong@pluto2:~/public_html/cs471/lecture/hpc$ cat test-openmpi-2.c
```

```
#include <mpi.h>
```

```
#include <stdio.h>
```

```
int main(int argc, char **argv) {
```

```
    int me, np, q, sendto;
```

```
    MPI_Status status;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &np);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &me);
```

```
    if (np%2==1) return 0;
```

```
    if (me%2==1) {
```

```
        sendto = me-1;}
```

```
    else {
```

```
        sendto = me+1;}
```

```
    if (me%2 == 0) {
```

```
        MPI_Send(&me, 1, MPI_INT, sendto, me, MPI_COMM_WORLD);
```

```
        MPI_Recv(&q, 1, MPI_INT, sendto, sendto, MPI_COMM_WORLD, &status);
```

```
    } else {
```

```
        MPI_Recv(&q, 1, MPI_INT, sendto, sendto, MPI_COMM_WORLD, &status);
```

```
        MPI_Send(&me, 1, MPI_INT, sendto, me, MPI_COMM_WORLD);
```

```
    }
```

```
    printf("Sent %d to proc %d, received %d from proc %d\n", me, sendto, q, sendto);
```

```
    MPI_Finalize();
```

```
    return 0;
```

```
}dong@pluto2:~/public_html/cs471/lecture/hpc$
```

Compiling and Running MPI Programs

`mpicc test-openmpi.c -o test-openmpi`

- Use the MPI C compiler (mpicc)
- Compile the program file test-openmpi.c
- Create an executable named test-openmpi

`mpirun -np 4 ./test-openmpi`

- mpirun → runs an MPI program.
- -np 4 → run it using 4 processes (4 copies of the program working together).
- ./test-openmpi → this is the program to run.

Computing π in Parallel

- Ratio of area of square to area of inscribed circle proportional to π

Imagine dartboard with
circle of radius R inscribed
in square

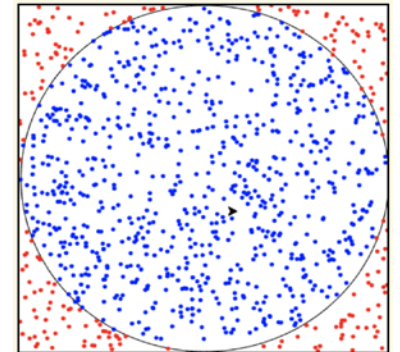
$$\text{Area of circle} = \pi R^2$$

$$\text{Area of square} = (2R)^2 = 4R^2$$

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi R^2}{4R^2} = \frac{\pi}{4}$$

$$\text{Fraction inside circle} \approx \frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi}{4}$$

$$\pi \approx 4 \times \frac{\text{Number of points inside circle}}{\text{Total points}}$$



- **What tasks independent of each other?**

Divide up number of dart throws evenly between processors, so each processor does a share of work

- **What tasks must be performed sequentially?**

Processor 0 will receive tallies from all the other processors, and will compute final value of π

```

/* Compute pi in serial */
#include "lcgenerator.h"
#include <stdio.h>
static long num_trials = 10000000;

int main(int argc, char **argv) {
    long Ncirc = 0;
    double pi, x, y;
    double r = 1.0; // radius of circle
    double r2 = r*r;

    for (long i = 0; i < num_trials; i++) {
        x = lcgrandom();
        y = lcgrandom();
        if ((x*x + y*y) <= r2)
            Ncirc++;
    }

    pi = 4.0 * ((double)Ncirc)/((double)num_trials);

    printf("\n \t Computing pi in serial: \n");
    printf("\t For %ld trials, pi = %f\n", num_trials, pi);
    printf("\n");

    return 0;
}

```

```

/* Random number generator -- and not a very good one, either! */

static long MULTIPLIER = 1366;
static long ADDEND = 150889;
static long PMOD = 714025;
long random_last = 0;

/* This is not a thread-safe random number generator */

double lcgrandom() {
    long random_next;
    random_next = (MULTIPLIER * random_last + ADDEND)%PMOD;
    random_last = random_next;

    return ((double)random_next/((double)PMOD));
}

```

```
dong@pluto2:~/public_html/cs471/lecture/hpc$ cat darts-mpi-v1.c
```

```
#include <mpi.h>
#include <stdio.h>

static long num_trials = 1000000;
static long MULTIPLIER = 1366;
static long ADDEND = 150889;
static long PMOD = 714025;

/* Thread-safe LCG random number generator using rank-specific seed */
double lcgrandom_r(unsigned int *seed) {
    long random_next;
    random_next = (MULTIPLIER * (*seed) + ADDEND) % PMOD;
    *seed = random_next;
    return ((double)random_next / (double)PMOD);
}

int main(int argc, char **argv) {
    long i, Ncirc = 0;
    double pi, x, y;

    int rank, size, j, manager = 0;
    MPI_Status status;
    long my_trials, temp;
```

```
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* Determine number of trials for this rank */
    my_trials = num_trials / size;
    if (num_trials % (long)size > (long)rank) my_trials++;

    /* Initialize rank-specific seed */
    unsigned int seed = rank;

    for (i = 0; i < my_trials; i++) {
        x = lcgrandom_r(&seed);
        y = lcgrandom_r(&seed);

        if ((x*x + y*y) <= 1.0)
            Ncirc++;
    }

    /* Reduce results to manager */
    if (rank == manager) {
        for (j = 1; j < size; j++) {
            MPI_Recv(&temp, 1, MPI_LONG, j, j, MPI_COMM_WORLD, &status);
            Ncirc += temp;
        }
        pi = 4.0 * ((double)Ncirc) / ((double)num_trials);
        printf("\n \t Computing pi using six basic MPI functions: \n");
        printf("\t For %ld trials, pi = %f\n\n", num_trials, pi);
    } else {
        MPI_Send(&Ncirc, 1, MPI_LONG, manager, rank, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}dong@pluto2:~/public_html/cs471/lecture/hpc$ |
```

MPI Collectives

- What if one task wants to send to *everyone*?

```
if (mytid == 0) {  
    for (tid=1; tid<ntids; tid++) {  
        MPI_Send( (void*)a, /* target= */ tid, ... );  
    }  
} else {  
    MPI_Recv( (void*)a, 0, ... );  
}
```

- Collective operations
 - Broadcast
 - Gather
 - Scatter
 - Reduce
 - All-
 - Barrier

Gather

- All processes need to send same (similar) message to manager
 - Could implement with each process calling MPI_Send(...) and manager looping through MPI_Recv(...)
 - Instead, use gather operation – more efficient, faster
 - Messages concatenated in rank order
- `int MPI_Gather(void* sendbuf, int sendcount, MPI_Datatype sendtype, void* recvbuf, int recvcount, MPI_Datatype recvttype, int root, MPI_Comm comm)`
- Note: `recvcount` = # items received from each process, not total

Reduce

- Perhaps we need to do sum of many subsums owned by all processors
- Perhaps we need to find maximum value of variable across all processors
- Perform global reduce operation across all group members
- `int MPI_Reduce(void* sendbuf, void* recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)`

sendbuf → your process's local data

recvbuf → where root process stores the reduced result

count → number of elements per process

datatype → type of each element

op → operation to combine elements

root → rank of the process that gets the result

comm → group of processes participating


```

#include <mpi.h>
#include <stdio.h>

static long num_trials = 1000000;
static long MULTIPLIER = 1366;
static long ADDEND = 150889;
static long PMOD = 714025;

/* Thread-safe LCG random number generator using rank-specific seed */
double lcgrandom_r(unsigned int *seed) {
    long random_next;
    random_next = (MULTIPLIER * (*seed) + ADDEND) % PMOD;
    *seed = random_next;
    return ((double)random_next / (double)PMOD);
}

int main(int argc, char **argv) {
    long i, Ncirc = 0;
    double pi, x, y;

    int rank, size;
    long my_trials;

```

```

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* Determine number of trials for this rank */
    my_trials = num_trials / size;
    if (num_trials % (long)size > (long)rank) my_trials++;

    /* Initialize rank-specific seed */
    unsigned int seed = rank;

    for (i = 0; i < my_trials; i++) {
        x = lcgrandom_r(&seed);
        y = lcgrandom_r(&seed);

        if ((x*x + y*y) <= 1.0)
            Ncirc++;
    }

    /* Reduce Ncirc from all ranks to the manager (rank 0) */
    long total_Ncirc = 0;
    MPI_Reduce(&Ncirc, &total_Ncirc, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        pi = 4.0 * ((double)total_Ncirc) / ((double)num_trials);
        printf("\n \t Computing pi using MPI_Reduce: \n");
        printf("\t For %ld trials, pi = %f\n\n", num_trials, pi);
    }

    MPI_Finalize();
    return 0;
}

```

Collective

Reduction Operations

Name	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical and
MPI_BAND	Bit-wise and
MPI_LOR	Logical or
MPI_BOR	Bit-wise or
MPI_LXOR	Logical xor
MPI_BXOR	Logical xor
MPI_MAXLOC	Max value and location
MPI_MINLOC	Min value and location

References

- Message Passing Interface (MPI) Tutorial: <https://hpc-tutorials.llnl.gov/mpi/>
- MPI Standard at MPI Forum: <https://www.mpi-forum.org/docs/>
- CAC Virtual workshop: <https://cvw.cac.cornell.edu/topics>